Reprojecting & I/O

Alexander Abuabara

DAEN 489 ISEN 489 Spring 2026

ENGINEERING
TEXAS A&M UNIVERSITY

Project Proposal Guidelines

The project proposal is the roadmap for a project worth 30% of your total grade.

- Successful proposals will lead to a successful final report.
- Ensure the document is professional as presentation quality (organization and clarity) is part of the grading criteria!.

Ideas/Examples

- Smart City: Digital Twin for Urban Heat Mitigation
  - Identify "heat islands" in a specific city and propose optimal locations for new green infrastructure
- Demand-Driven Logistics: Optimizing "Dark Store" Locations
  - Dark stores are micro-fulfillment centers, minimize delivery times during peak business hours.
- Disaster Response: Flood Risk for Critical Infrastructure
  - Evaluate the vulnerability of emergency services (hospitals, fire stations) during a 100-year flood event.
- *Moogle Gaps* https://mooglegaps.netlify.app/
- *Radio Garden* https://radio.garden/

Project Proposal Guidelines
Purpose

- The project proposal is the first formal step in your applied spatial analysis project.
- In groups of 2, you will design a solution to a practical engineering problem using spatial data engineering techniques.
- The proposal "ensures" your team has a viable plan for data integration, analysis, and visualization.
- Concise document (submitted on Canvas) covering the following parts:
  - Problem Statement
  - Data Sources & Integration
  - Methodology
  - Analysis Plan
  - Visualization Goals

Project Proposal Guidelines
Required Components

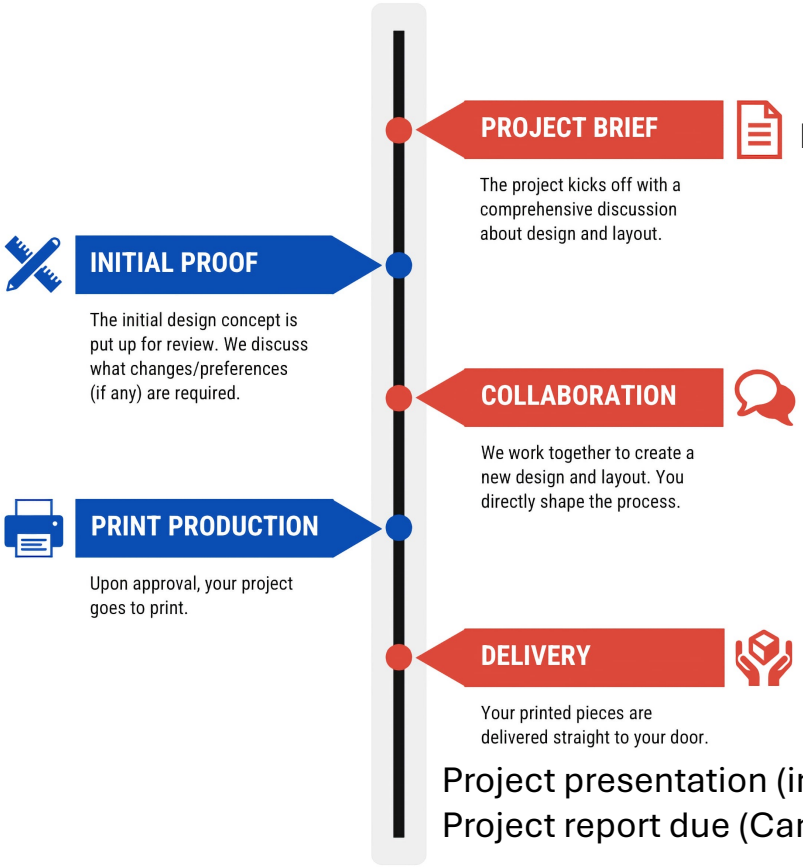
- **Problem Statement**:
  - Define a practical engineering problem that can be addressed through spatial analysis.
- **Data Sources & Integration**:
  - Identify the "large-scale" spatial datasets you plan to use.
  - Describe how you will perform Spatial ETL (Extract, Transform, Load), including handling coordinate reference systems (CRS).
- **Methodology**:
  - Outline the spatial and attribute operations required.
  - Explain how you will model data structures to reflect spatial relationships.
  - If applicable, describe plans for Raster-Vector interaction or Data Cube construction.
- **Analysis Plan**:
  - Describe the specific spatial statistics or modeling techniques (e.g., interpolation, spatial regression, or suitability analysis) you intend to use.
- **Visualization Goals**:
  - Briefly describe how you plan to communicate results (e.g., report with high-resolution maps or a Story Map).

Project Proposal Guidelines
Expectations & Standards

- **Reproducibility**:
  - Your final project must include an automated workflow and reproducible code.
  - Use the proposal to identify the tools (e.g., R, Python, or Docker) and libraries you will use to ensure this.
- **Ethics & Privacy**:
  - You must briefly note any licensing or ethical considerations related to your chosen datasets.
- **Academic Integrity**:
  - All work must reflect your team's independent effort.
  - While AI can be used for brainstorming or clarifying your writing, it cannot be used to generate the substantive analysis or solve the broad programming tasks of the project.

PLAN SMART

PROJECT DELIVERY TIMELINE



Project proposal due 4/9

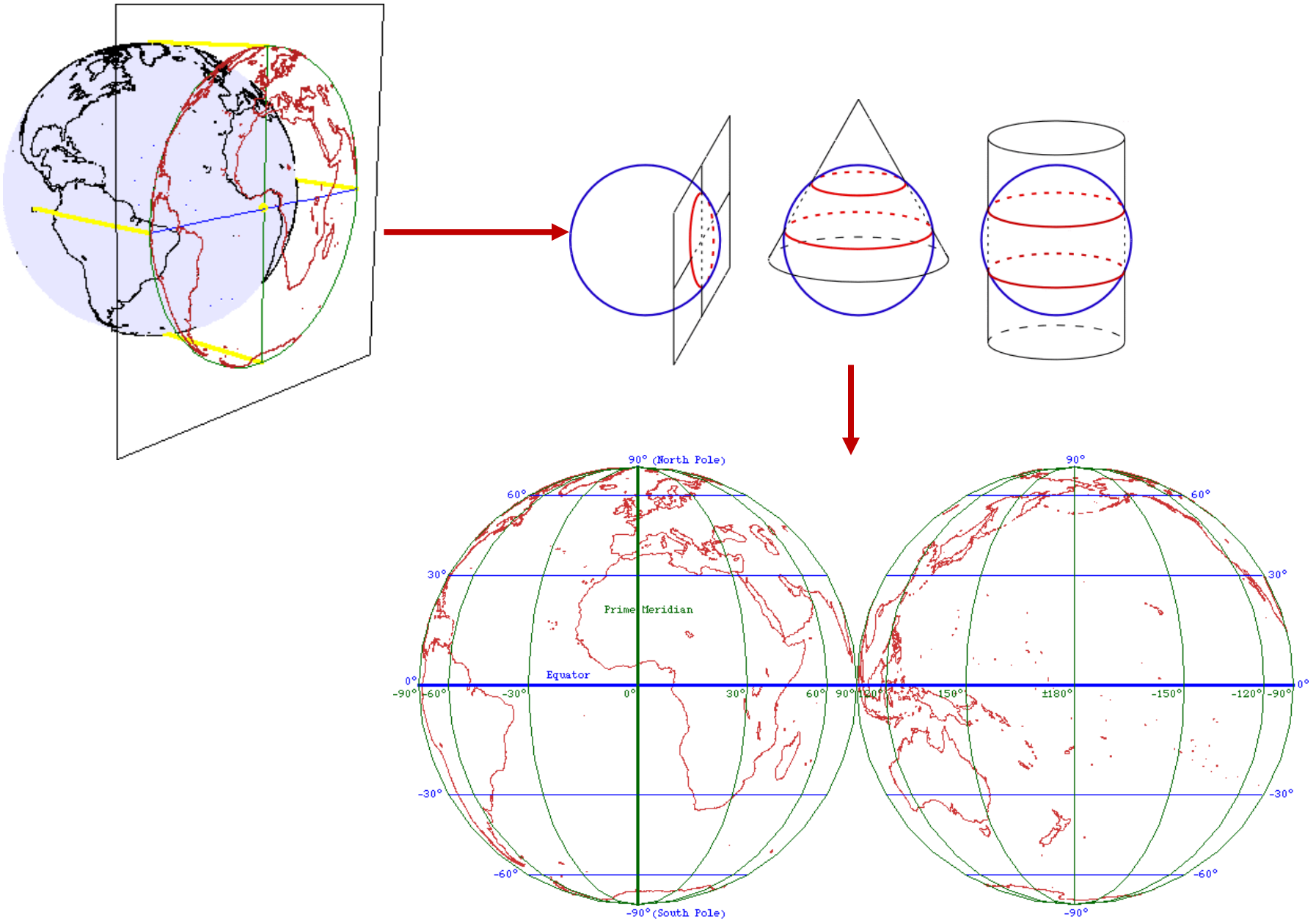
Project presentation (in class) 4/23

Project report due (Canvas, no class meeting) 4/30

Reprojecting geographic data

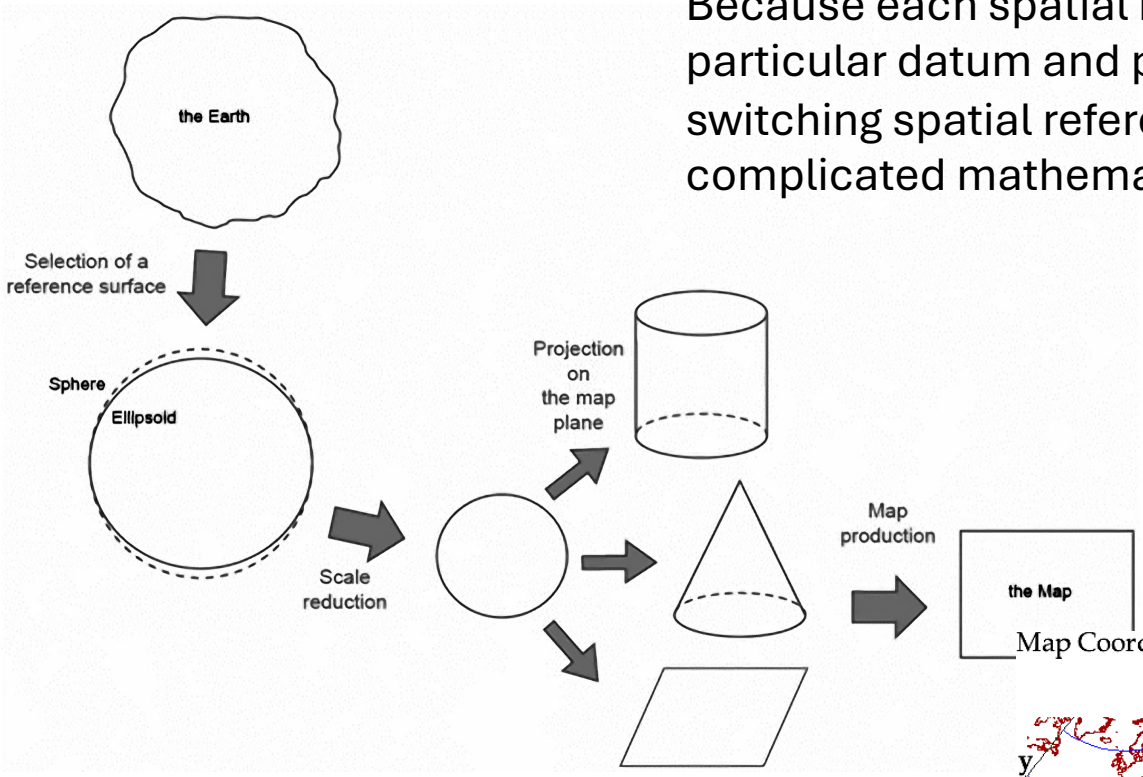
Refs.:

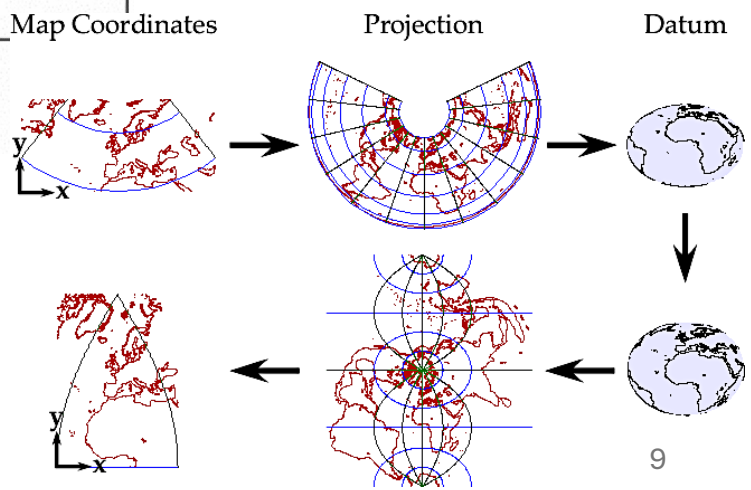
- https://www.ats.amherst.edu/software/gis/mapping_coordinate_data/
- https://colorado.pressbooks.pub/makingmaps/chapter/chapter-9-datums-coordinate-systems-and-map-projections/


90° (North Pole)
60°
30°
Prime Meridian
Equator
0°
-30°
-60°
-90°(South Pole)
-90°
-60°
-30°
0°
30°
60°
90°
120°
150°
±180°
-150°
-120°
-90°
90°
60°
30°
0°
-30°
-60°
-90°

To simultaneously display two or more sets of GIS data with different spatial references, some of them must be recast to a common spatial reference.

Because each spatial reference is based on a particular datum and possibly also a projection, switching spatial references can involve a complicated mathematical process:





Switching datums is generally more complicated than simply unprojecting and reprojecting, so approximations are usually made that can introduce small errors.

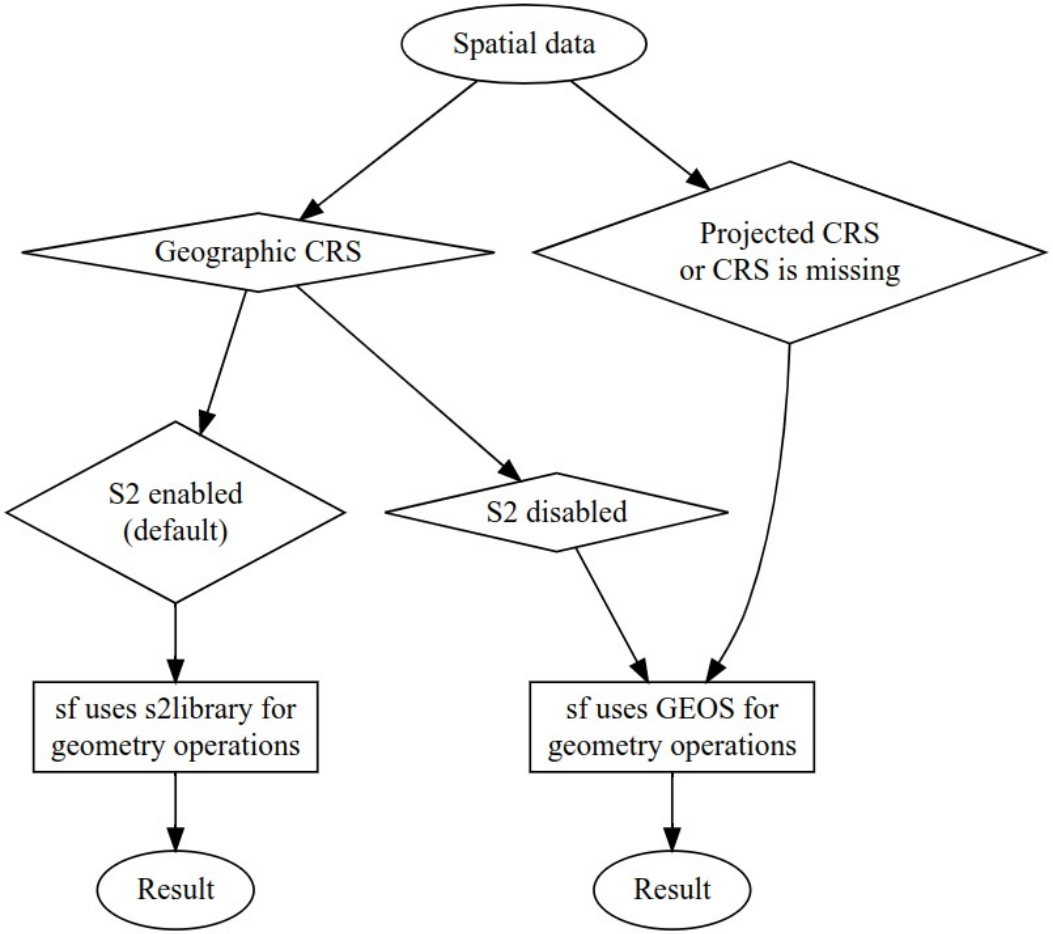
Reprojecting https://r.geocompx.org/reproj-geo-data Geometry Operations & CRS

- The "Orange Peel" Problem
  - Why can't we represent a 3D Earth on a 2D screen without distortion?
  - CRS Components
    - GCS (Geographic): Units in degrees (longitude/latitude)
    - PCS (Projected): Units in linear measurements (meters/feet)
- Transformations in R
  - Using **st_crs()** to check and **st_transform()** to project
  - Never use **st_set_crs()** to change a projection
    → use it only to define a missing one!
  - Affine Transformations
    - Shifting, scaling, and rotating geometries for non-standard data alignment (e.g., lidar/sonar).

```
st_crs("EPSG:4326")
#> Coordinate Reference System:
#>   User input: EPSG:4326
#>   wkt:
#> GEOGCRS["WGS 84",
#>     ENSEMBLE["World Geodetic System 1984 ensemble",
#>         MEMBER["World Geodetic System 1984 (Transit)"],
#>         MEMBER["World Geodetic System 1984 (G730)"],
#>         MEMBER["World Geodetic System 1984 (G873)"],
#>         MEMBER["World Geodetic System 1984 (G1150)"],
#>         MEMBER["World Geodetic System 1984 (G1674)"],
#>         MEMBER["World Geodetic System 1984 (G1762)"],
#>         MEMBER["World Geodetic System 1984 (G2139)"],
#>         ELLIPSOID["WGS 84",6378137,298.257223563,
#>             LENGTHUNIT["metre",1]],
#>         ENSEMBLEACCURACY[2.0]],
#>     PRIMEM["Greenwich",0,
#>         ANGLEUNIT["degree",0.0174532925199433]],
#>     CS[ellipsoidal,2],
#>         AXIS["geodetic latitude (Lat)",north,
#>             ORDER[1],
#>             ANGLEUNIT["degree",0.0174532925199433]],
#>         AXIS["geodetic longitude (Lon)",east,
#>             ORDER[2],
#>             ANGLEUNIT["degree",0.0174532925199433]],
#>     USAGE[
#>         SCOPE["Horizontal component of 3D system."],
#>         AREA["World."],
#>         BBOX[-90,-180,90,180]],
#>     ID["EPSG",4326]]
```

The WKT representation of the WGS84 CRS, which has the identifier EPSG:4326.

Behavior of the geometry operations in the **sf** package depending on the input data's CRS



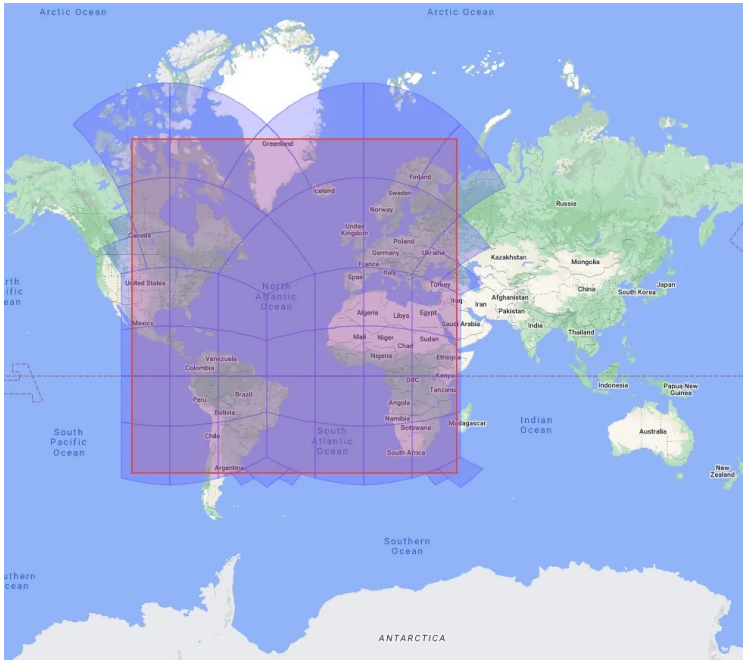
S2 Geometry Library
https://s2geometry.io

... represents
all data on a
3D sphere

Core Difference: Flat vs. Round

Comparison

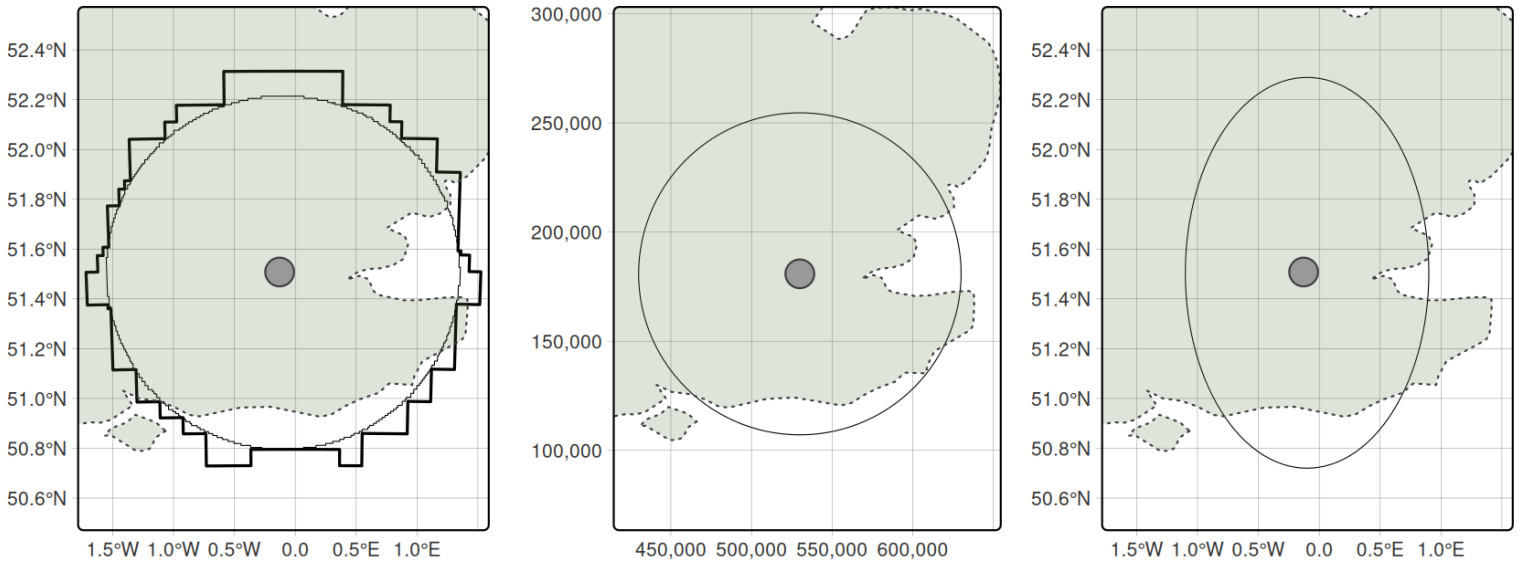
Feature	s2 (Spherical)	GEOS (Planar)
Model	3D Sphere	2D Flat Plane
Best For	Geographic data (Lon/Lat)	Projected data (UTM, State Plane)
Distance	Great circle (Spherical)	Straight line (Pythagorean)
Performance	Highly optimized for global scales	Generally very fast
Edge Cases	Handles the Date Line & Poles natively	Struggles with Date Line (180°)
R	sf_use_s2(TRUE) *default since sf 1.0*	sf_use_s2(FALSE)



Google S2: On a globe projection the lines are skewed

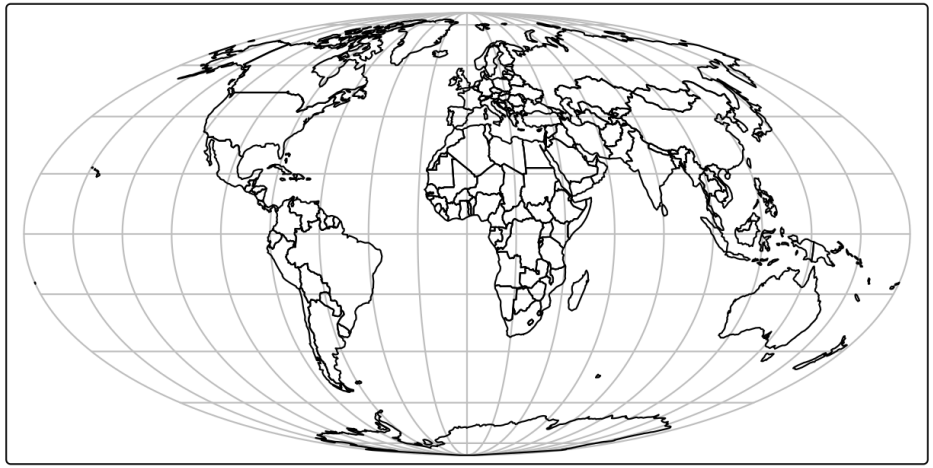
https://medium.com/@claude.ducharme/selecting-a-geo-representation-81afeaf3bf01

- Use s2 for Unprojected Data
  - **sf_use_s2(TRUE) # using s2**
  - If your data is in WGS84 (EPSG: 4326), keep s2 turned on.
    - It prevents the "long/lat is not planar" warnings and provides accurate global measurements.
  - Buffer Example: A 100km buffer at 60°N latitude will look like an ellipse on a flat map but is a perfect circle on the Earth's surface. s2 handles this correctly.
- Use GEOS for Projected Data
  - **sf_use_s2(FALSE) # using GEOS**
  - If you have already transformed your data into a local projection (like a specific UTM zone), sf automatically bypasses s2 and uses GEOS.
    - GEOS is the gold standard for high-precision, small-area engineering and surveying tasks where the Earth's curvature is negligible.
- Validity
  - **GEOS** and **s2** have different definitions of a "valid" polygon.
  - A shape that passes a validity check in GEOS might fail in s2 because of how edges are interpolated on a sphere.
  - If you get errors, **st_make_valid()** is your best friend :)

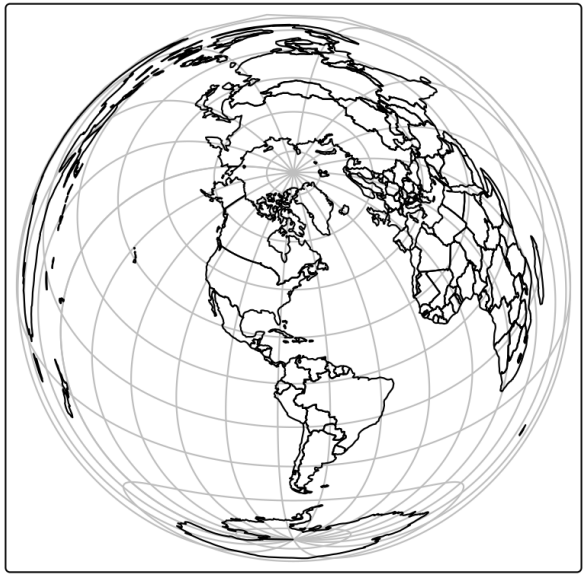


- Buffers around London showing results created with the S2 spherical geometry engine on lon/lat data (left), projected data (middle) and lon/lat data without using spherical geometry (right).
- The left plot illustrates the result of buffering unprojected data with sf, which calls Google's S2 spherical geometry engine by default with max cells set to 1000 (thin line).
- The thick, blocky line illustrates the result of the same operation with max cells set to 100.

Mollweide projection of the world.
world_mollweide = st_transform(world, crs = "+proj=moll")



Winkel tripel projection of the world.
world_wintri = st_transform(world, crs = "+proj=wintri")



Lambert azimuthal equal-area projection of the world centered on New York City.
world_laea2 = st_transform(world, crs = "+proj=laea +x_0=0 +y_0=0 +lon_0=-74 +lat_0=40")

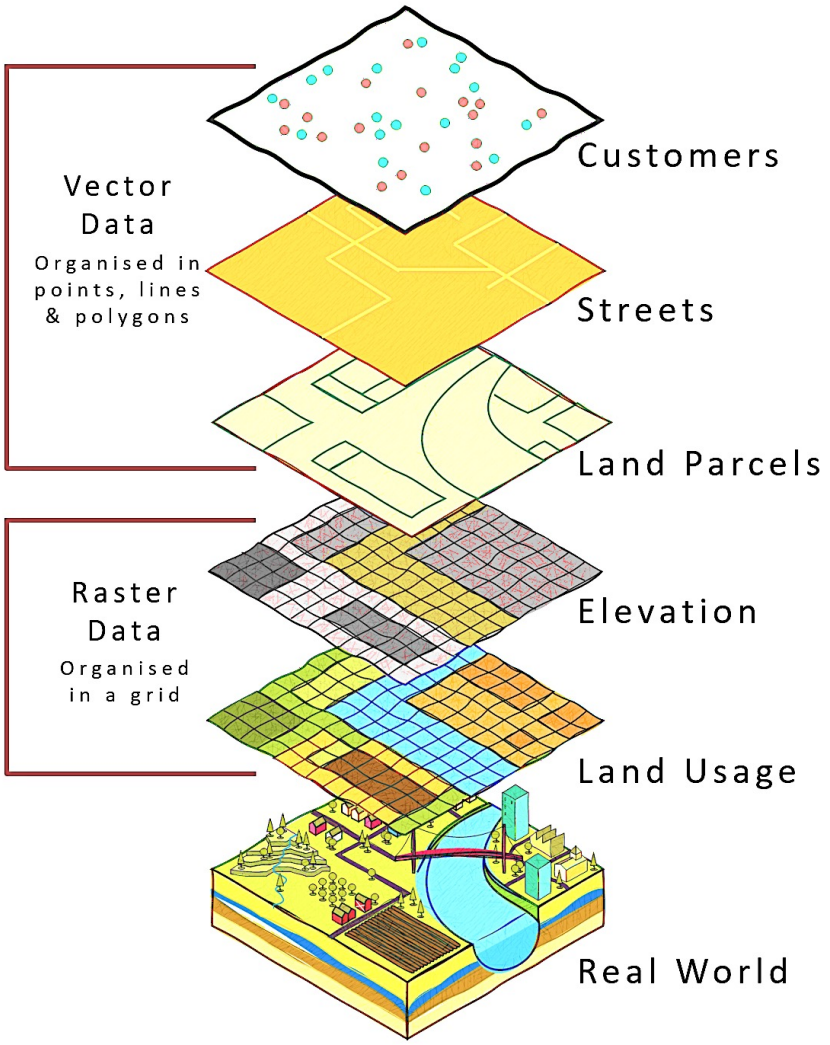
Querying and setting coordinate systems

- The CRS can be retrieved with the sf function **st_crs()**.
- The **st_crs** function also has one helpful feature – we can retrieve some additional information about the used CRS.
- For example, try to run:
  - **st_crs(new_vector)$IsGeographic** to check if the CRS is geographic or not
  - **st_crs(new_vector)$units_gdal** to find out the CRS units
  - **st_crs(new_vector)$srid** to extract its 'SRID' identifier (when available)
  - **st_crs(new_vector)$proj4string** to extract the proj-string representation
- In cases when a CRS is missing or the wrong CRS is set, the **st_set_crs()** function can be used (in this case the WKT string remains unchanged because the CRS was already set correctly when the file was read-in).
- The same function, **crs()**, can be also used to set a CRS for raster objects.

When to reproject?

- In real-world applications, however,
  CRSs are usually set automatically when data is read-in.
- In many projects the main CRS-related task is to transform objects, from one CRS into another.
- But when should data be transformed? And into which CRS?
  - There are no clear-cut answers to these questions and CRS selection always involves trade-offs. Maling, D. H. 1992. Coordinate Systems and Map Projections
- There are some general principles that can help you decide when to transform:
  - In some cases, transformation to a geographic CRS is essential, such as when publishing data online with the leaflet package.
  - Or when two objects with different CRSs must be compared or combined.
- Which CRS to use?
  - Tricky, and there is rarely a 'right' answer.
    - Geographic CRSs, the answer is often WGS84
    - Projected CRS, in most cases it is not something that we are free to decide: "often the choice of projection is made by a public mapping agency".

Geographic data I/O



Geographic data I/O https://r.geocompx.org/read-write

Common Spatial Data Formats

Name	Extension	Information	Type	Model
ESRI Shapefile	.shp	Popular format consisting of at least three files. No support for: files > 2 GB; mixed types; names > 10 chars; cols > 255.	Vector	Partially open
GeoJSON	.geojson	Extends JSON by including a subset of simple feature representation; mostly used for long/lat coordinates; extended by TopoJSON.	Vector	Open
KML	.kml	XML-based format for spatial visualization (Google Earth). Zipped KML forms the KMZ format.	Vector	Open
GPX	.gpx	XML schema specifically created for the exchange of GPS data.	Vector	Open
FlatGeobuf	.fgb	Single file format for quick reading/writing. Includes streaming capabilities.	Vector	Open
GeoTIFF	.tif / .tiff	Popular raster format. A TIFF file containing additional spatial metadata.	Raster	Open
Arc ASCII	.asc	Text format; first six lines are the header, followed by cell values in rows and columns.	Raster	Open
SQLite/SpatiaLite	.sqlite	Standalone relational database; SpatiaLite is the spatial extension of SQLite.	Vector & Raster	Open
ESRI FileGDB	.gdb	Created by ArcGIS. Allows multiple feature classes and topology. Limited support from GDAL.	Vector & Raster	Proprietary
GeoPackage	.gpkg	Lightweight database based on SQLite; easy, platform-independent exchange.	Vector & Raster*	Open

Modern Vector Formats

- Shapefiles are 1990s tech
  - DBF, 8-char names, 2GB limit, multi-file, etc.
- The Solutions
  - **GeoPackage (.gpkg)**
    - SQLite-based, single file.
    - Supports multiple layers and non-spatial tables.
  - **Web Standard (GeoJSON)**
    - Best for web APIs
    - But slow for large datasets (larger file size than GPKG, text-based: easy to inspect, slower for big data)
  - **Cloud Standard (GeoParquet)**
    - Columnar storage for massive scale
    - Great for data engineering workflows

```
# GeoPackage is often the best
  default (fast, compact, supports
  multiple layers)
st_write(data, "data.gpkg",
  delete_dsn = TRUE)


# To specify layer name
st_write(data, "data.gpkg", layer =
  "my_layer", delete_layer = TRUE)


# save as GeoJSON (.geojson)
st_write(data, "data.geojson",
  driver = "GeoJSON")


data_parquet <- data %>%
  mutate(geometry =
  st_as_binary(geometry))
```

Modern Vector Formats

- **Cloud Standard (GeoParquet)**
  - Columnar storage for massive scale
  - Great for data engineering workflows
  - sf objects are not always fully preserved automatically
  - Best practice: convert geometry explicitly

Or use the newer spatial support

https://arrow.apache.org/docs/r/

But still working in progress

https://medium.com/radiant-earth-insights/geoparquet-parquet-geospatial-types-a-time-of-transition-a42e391cdab2

```
library(arrow)

write_parquet(data_parquet,
  "data.parquet")
```

Out-of-Memory Processing (Rasters)

- The **terra** advantage
  - Unlike vectors, terra rasters don't load the whole file into RAM by default.
  - Pointers: The R object is just a pointer to the file on disk.
  - Chunking: Functions like app() or predict() process data in blocks to prevent system crashes.
  - Format Matters: Use Cloud-Optimized GeoTIFFs (COG) to read only the pixels you need for the current view.

Raster data

Very efficient!

```
snow = rast(myurl)


rey =
  data.frame(lon = -21.94,
             lat = 64.15)


snow_rey = extract(snow, rey)
```

Input/Output Backbone

- **Vector Formats**
  - Beyond the Shapefile:
    - GeoPackage (**.gpkg**) is the modern standard (single file, SQLite-based)
    - GeoJSON for web applications
- **Raster Formats**
  - GeoTIFF and cloud-optimized variants (COG)
- **R Workflow**
  - **st_read()** and **st_write()** from the sf package
  - **rast()** and **writeRaster()** from the terra package
  - Database Integration
    - Brief overview of connecting to PostGIS
    - PostGIS is an extension of PostgreSQL that turns it into a full spatial database
    - References:
      - https://journal.r-project.org/articles/RJ-2018-025/
      - https://postgis.net/documentation/getting_started/
      - https://youtu.be/FKzw9HJ6XYA?si=U7agrt_By-Fz7DTK
      - https://youtu.be/9pOCVjR7iaA?si=9kP-U9uWaqbbXLq0

Geographic metadata

- Geographic metadata are a cornerstone of geographic information management, used to describe datasets, data structures and services.
- They help make datasets FAIR (Findable, Accessible, Interoperable, Reusable) and are defined by the ISO/OGC standards, in particular the ISO 19115 standard and underlying schemas.
- These standards are widely used within spatial data infrastructures, handled through metadata catalogs.
- Geographic metadata can be managed with **geometa**, a package that allows writing, reading and validating geographic metadata according to the ISO/OGC standards.
    - It already supports various international standards for geographic metadata information, such as the ISO 19110 (feature catalogue), legacy ISO 19115-1 and 19115-2 (geographic metadata for vector and gridded/imagery datasets), ISO 19115-3, ISO 19119 (geographic metadata for service), and ISO 19136 (Geographic Markup Language) providing methods to read, validate and write geographic metadata from R using the ISO/TS 19139 (XML) and ISO 19115-3 technical specifications.

```
library(geometa)
# create a metadata
md = ISOMetadata$new()
#... fill the metadata 'md' object
# validate metadata
md$validate()
# XML representation of the ISOMetadata
xml = md$encode()
# save metadata
md$save("my_metadata.xml")
# read a metadata from an XML file
md = readISO19139("my_metadata.xml")
```

The package comes with more examples (https://github.com/eblondel/geometa/tree/master/inst/extdata/examples) and has been extended by packages such as **geoflow** (https://github.com/r-geoflow/geoflow) to ease and automate the management of metadata.

Geographic Web Services

- Effort to standardize web APIs for accessing spatial data
  - ISO/OGC Spatial Schema (ISO 19107:2019) https://www.iso.org/standard/66175.html
  - Simple Features (ISO 19125-1:2004) https://www.iso.org/standard/40114.html
  - Format data (Geographic Markup Language GML) https://www.iso.org/standard/75676.html

Data input (I)

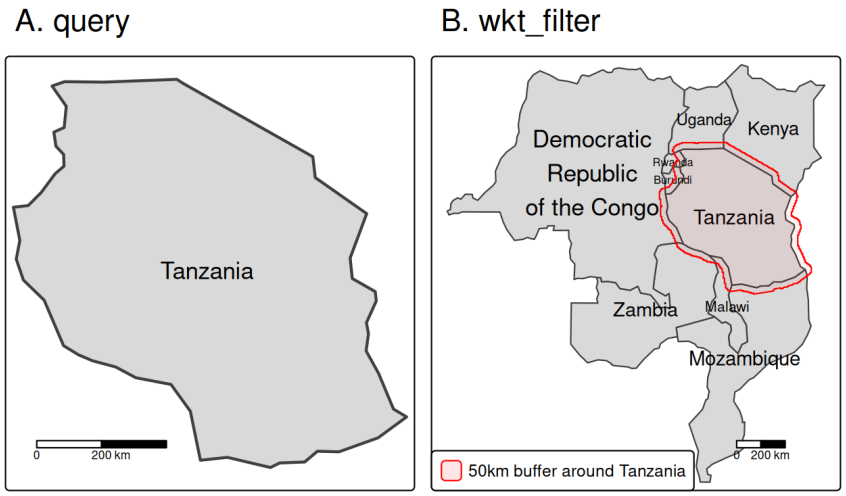
Vector data

- Spatial vector data comes in a wide variety of file formats.
- Most popular representations such as **.geojson** and **.gpkg** files can be imported directly into R with the sf function **read_sf()** (or the equivalent **st_read()**), which uses GDAL's vector drivers behind the scenes.
- The **st_drivers()** function returns a data frame containing name and long_name in the first two columns, and features of each driver available to GDAL (and therefore sf).

Raster data

- Similar to vector data, raster data comes in many file formats with some supporting multi-layer files. terra's **rast()** command reads in a single layer when a file with just one layer is provided.

Vector data



Reading a subset of the vector data using (A) a query and (B) a wkt filter.

```
tanzania = read_sf(f, query =
  'SELECT * FROM world WHERE
  name_long = "Tanzania"')


tanzania_buf = st_buffer(tanzania,
  50000)

tanzania_buf_geom =
  st_geometry(tanzania_buf)

tanzania_buf_wkt =
  st_as_text(tanzania_buf_geom)
```

Data output (O)

Vector data

- The counterpart of read_sf() is write_sf()

  write_sf(obj = world, dsn = "world.gpkg")

  Notes:

  Instead of overwriting the file, can add a new layer to the file by specifying the layer argument

  write_sf(obj = world, dsn = "world_many_layers.gpkg", layer = "second_layer")

  Use AS_XY for simple point datasets (it creates two new columns for coordinates) or AS_WKT for more complex spatial data:

  layer_options = "GEOMETRY=AS_XY" or **layer_options = "GEOMETRY=AS_WKT"**

Raster data

- The terra package offers seven data types when saving a raster: INT1U, INT2S, INT2U, INT4S, INT4U, FLT4S, and FLT8S,44 which determine the bit representation of the raster object written to disk. (see table)
  - Which data type to use depends on the range of the values of your raster object.
  - The more values a data type can represent, the larger the file will get on disk.
  - Unsigned integers (INT1U, INT2U, INT4U) are suitable for categorical data, while float numbers (FLT4S and FLT8S) usually represent continuous data

```
writeRaster(single_layer, filename = "my_raster.tif", datatype = "INT2U")
```

- To change or disable the compression, modify the argument:

```
writeRaster(x = single_layer, filename = "my_raster.tif",
                              gdal = c("COMPRESS=NONE"), overwrite = TRUE)
```

Geographic data packages

Package	Description
climateR	Access over 100,000 gridded climate and landscape datasets from over 2,000 data providers by area of interest.
elevatr	Access point and raster elevation data from various sources.
FedData	Datasets maintained by the US federal government, including elevation and land cover.
geodata	Download and import imports administrative, elevation, WorldClim data.
osmdata	Download and import small OpenStreetMap datasets.
osmextract	Download and import large OpenStreetMap datasets.
rnaturalearth	Access Natural Earth vector and raster data.
rnoaa	Import National Oceanic and Atmospheric Administration (NOAA) climate data.

Selected R packages for geographic data retrieval.

Google Earth Engine

- Traditional GIS workflows involve downloading massive datasets (TB/PB scale), managing local storage, and hitting hardware bottlenecks during processing.
- GEE is a Platform as a Service (PaaS) that co-locates a multi-petabyte catalog of satellite imagery with a high-performance, parallelized computation service.
- Data Engineering Perspective:
  - Data Lake: 40+ years of historical data (Landsat, Sentinel, MODIS, Climate data).
  - Computation: Distributed processing across Google's data centers.
  - Abstraction: You don't manage the clusters; you write the logic (JavaScript/Python), and GEE handles the "embarrassingly parallel" distribution of pixels.

Google Earth Engine
Spatial Data Pipeline

- Data Model:
  - Images/ImageCollections: Raster data (multidimensional arrays/tensors).
  - Features/FeatureCollections: Vector data (points, lines, polygons).
- Processing Engine: Uses a Lazy Evaluation model, i.e., computations are only executed when a result (like a map tile or a chart) is explicitly requested.
- Key Operations:
  - **Filtering**: Metadata-based querying (Space, Time, Cloud cover).
  - **Mapping**: Applying a function over an entire ImageCollection (functional programming).
  - **Reducing**: Spatial and temporal aggregations (e.g., calculating the mean NDVI of a province over 10 years).
  - **Integration**: Seamless export to Google Cloud Storage, BigQuery, or AI Platform for downstream Machine Learning.

Google Earth Engine **Example**

Earth Engine Notebook Client

- GEE is not a traditional database. It's an image processing engine. (Javascript, Python, R)
  - While it handles vectors well, its true "superpower" is the ability to run a computation over 10,000 images simultaneously without the user ever worrying about RAM or CPU allocation.

A very simple example …

```python
import ee
import geemap

# Login/Initialize the Earth Engine module
ee_Authenticate()
ee.Initialize()

# 1. DATA INGESTION: Access the Sentinel-2 Surface Reflectance collection
s2_collection = (ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterBounds(ee.Geometry.Point([-122.2, 37.8])) # Filter by Location (Spatial)
    .filterDate('2023-01-01', '2023-12-31')         # Filter by Time (Temporal)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))) # Metadata Filter

# 2. TRANSFORMATION: Define a function to calculate NDVI
def calculate_ndvi(image):
    ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI')
    return image.addBands(ndvi)

# 3. MAPPING: Apply the function over the entire collection (Parallel execution)
processed_collection = s2_collection.map(calculate_ndvi)

# 4. REDUCTION: Reduce 100+ images into one "Median" composite image
final_median = processed_collection.median()

print("Processing complete. The 'final_median' is a virtual object ready for export.")
```

```r
library(rgee)
library(magrittr)

# 1. Initialize the Earth Engine session
ee.Authenticate()
ee_Initialize()

# 2. DATA INGESTION: Sentinel-2 Surface Reflectance
# Note: In rgee, Python's '.' becomes '$'
s2_collection <- ee$ImageCollection('COPERNICUS/S2_SR_HARMONIZED')$
  filterBounds(ee$Geometry$Point(c(-122.2, 37.8)))$
  filterDate('2023-01-01', '2023-12-31')$
  filter(ee$Filter$lt('CLOUDY_PIXEL_PERCENTAGE', 10))

# 3. TRANSFORMATION: Function to calculate NDVI
calculate_ndvi <- function(image) {
  ndvi <- image$normalizedDifference(c('B8', 'B4'))$rename('NDVI')
  return(image$addBands(ndvi))
}

# 4. MAPPING & REDUCTION: Functional programming over the collection
final_median <- s2_collection$
  map(calculate_ndvi)$
  median()

# Print metadata to the console
ee_print(final_median)
```

Refs.: https://developers.google.com/earth-engine/tutorials/community/intro-to-python-api
https://github.com/giswqs/earthengine-py-notebooks
https://github.com/giswqs/earthengine-py-examples

Next

- R/Python supports many different static and interactive graphics.
- Next class we will cover map-making in detail.